1 Binary Logistic Regression

We are given a dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ with M-dimensional inputs $x \in \mathbb{R}^M$ and binary outputs $y \in \{0, 1\}$. We want to build a model as a linear combination of $x^{(i)}$ and weights followed by a nonlinear activation function $\alpha(.)$, s.t. the model predicts conditional probability $P(y^{(i)}|x^{(i)}, \theta)$.

Consider the form below:

$$\mathcal{D} := \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,M} \\ x_{2,1} & \ddots & & \\ \vdots & & \ddots & \\ x_{N,1} & & & x_{N,M} \end{bmatrix} \quad \hat{Y} := \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}$$

1.1 What are the dimensions of weight vector 0?

To obtain the form $\mathcal{D}\vec{\theta} = \hat{Y}$. Consider $dims(\mathcal{D}), dims(\hat{Y}) \implies (N \times M)$, $(N \times 1)$ respectively.

$$\therefore \ dims(\vec{\theta}) := (M \times 1)$$

Consider the form below:

$$\begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,M} \\ x_{2,1} & \ddots & & \\ \vdots & & \ddots & \\ x_{N,1} & & & x_{N,M} \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_M \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{bmatrix}$$

1.2. Choice of α

What activation is appropriate for this task? Justify your choice. Write the equation for $\alpha(x)$.

$$\forall \ y \in \hat{Y}, y \in [0, 1] \implies \forall \ x \in \mathbb{R}, \ \alpha(x) \in \{0, 1\}$$

$$\text{Consider: } \beta(x) := \frac{1}{1 + e^{-x}} \implies \forall \ x \in \mathbb{R}$$

$$\implies \forall \ x \in \delta(x) := u(\beta(x)), \ x \in \{0, 1\}$$

Apply delta-dirac function centered at 0.5 with the input x composed via a sigmoid function

$$\therefore \ \delta(x) \equiv \alpha(x)$$

1.3. Conditional Probability Equation

Write a single equation to express the conditional probability $P(y^{(i)}|x^{(i)}, \theta)$ in terms of $\alpha(.)$, θ.

$$\text{Let } P(y^{(i)}|x^{(i)}, \theta) := P(y_i|x_i, \theta) \equiv \text{"}\textit{The probability of } y_i \textit{ given } x_i, \ \theta.\text{"}$$

$$\text{Recall: } \begin{cases} a_1 = w_1 \cdot x + b_1 \\ a_2 = w_2 f_1(a_1) \\ a_3 = w_3 f_2(a_2) \\ y = f_3(a_3) \end{cases}$$

$$P(y_i|x_i,\theta) = (\alpha(\theta^T x_i))^{y_i}(1-\alpha(\theta^T x_i))^{1-y_i}$$

1.4. Likelihood Expression

We assume each observation of $y^{(i)}$ in the data is independent and identically distributed. This means we can write down the likelihood of the data as a product of negative conditional probabilities. Express and simplify this likelihood $J(\theta)$

$$\forall\ y, y \text{ is independent and identical } \implies \prod_{i=1}^{N} \left[\alpha(\theta^T x_i)\right]^{y_i} \left[1-\alpha(\theta^T x_i)\right]^{1-y_i}$$

$$\equiv \log J(\theta) = \sum_{i=1}^{N} \Big[y_i \log \alpha(\theta^T x_i) + (1-y_i)\log(1-\alpha(\theta^T x_i)) \Big]$$

Computationally More Efficent

1.5. Stochastic Gradient Descent

In stochastic gradient descent, we use only a single $x^{(i)}$ to compute the likelihood and then find the gradient as the partial derivative of $J(\theta)$ w.r.t. each j^{th} parameter θ_j Derive the expression for this partial derivative $\frac{\partial J^{(i)}(\theta)}{\partial \theta_j}$

Apply Dimensional Analysis

$$\frac{\partial J_i}{\partial \theta_j} = \frac{\partial J_i}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial(\theta^T x_i)} \cdot \frac{\partial(\theta^T x_i)}{\partial \theta_j} = \vec{x}(\hat{y} - \vec{y})$$

1.6. Gradient Interpretation

You may have noticed that the gradient computed above has a very simple form.

Part (A): *Express the gradient equation in simple English.*
The gradient is the landscape describe by the space of vectors which describe the differences impact of the difference between predicted values and ground truth with respect to the weights and biases associated. The directions of the vectors which describe the space show the directions which will maximize or minimize error, as maximas or minimas, respectively.

Part (B) *Assuming that one has performed a forward pass to obtain a prediction, how many computations (how many multiplications and how many additions/subtractions) would one need to do to obtain the gradient for each weight?*

Operation	**Number of Computations**
Compute $z = \theta^T x = \sum_{j=1}^{M} \theta_j x_j$	M multiplications, $M-1$ additions
Compute $\hat{y} = \alpha(z) = \frac{1}{1+e^{-z}}$	1 exponentiation, 2 additions/subtractions, 1 division
Compute gradient $\frac{\partial J}{\partial \theta} = (\hat{y} - y)\vec{x}$	1 subtraction, M multiplications

2 Logistic Regression Computations by Hand

Refer to code, in attached zip file for adjustments (see "main-ec.py")

A logistic regression model is given by the equation $\hat{y} = \sigma(\theta^T x)$ where is the sigmoid function. Consider a logistic regression task with inputs $x \in \mathbb{R}^3$ and outputs $y \in \{0, 1\}$. The dataset has three datapoints: {, ([1, 1, -1], 1), ([1, 2, 1], 0)}.

- The initial weight vector is $\hat{\theta} = [-2, 2, 1]$.

2.1. Training Objective $J(\theta)$

Assuming we're using mean-squared error as the training objective $J(\theta)$ write the equation for $J(\theta)$. Then compute the value of $J(\theta)$ with the initial weight vector .

$$J(\theta) = \frac{1}{2N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

$$\implies \frac{\partial J^{(i)}(\theta)}{\partial \theta_j} = (\hat{y}^i - y^i) \cdot y^i(1 - \hat{y^i}) \cdot x_j^i$$

$$\equiv \nabla_\theta J^{(i)} = (\hat{y^i} - y^i)\hat{y^i}(1 - y^i)x^i$$

2.2. Gradient Computation

Calculate the partial derivatives for the first training example: $\frac{\partial J^{(1)}(\theta)}{\partial \theta_1}$, $\frac{\partial J^{(1)}(\theta)}{\partial \theta_2}$, $\frac{\partial J^{(1)}(\theta)}{\partial \theta_3}$.

$$\begin{bmatrix}1 & 1 & -1\end{bmatrix}\begin{bmatrix}-2\\2\\1\end{bmatrix} = \begin{bmatrix}0.268\end{bmatrix} = \hat{y}_1$$

$$\hat{y_1} - y = 0.268 - 1 = -0.732$$

$$\hat{y_1}(1 - \hat{y_1}) = 0.268(0.731) = 0.195$$

$$\nabla_\theta J^{(1)} = (-0.732)(0.195)\begin{bmatrix}1\\1\\-1\end{bmatrix} = -0.142\begin{bmatrix}1\\1\\-1\end{bmatrix} = \begin{bmatrix}-0.143\\-0.143\\0.143\end{bmatrix}$$

Calculate the partial derivatives for the second training example: $\frac{\partial J^{(2)}(\theta)}{\partial \theta_1}$, $\frac{\partial J^{(2)}(\theta)}{\partial \theta_2}$, $\frac{\partial J^{(2)}(\theta)}{\partial \theta_3}$

$$\begin{bmatrix}1 & 2 & 1\end{bmatrix}\begin{bmatrix}-2\\2\\1\end{bmatrix} = \begin{bmatrix}0.952\end{bmatrix} = \hat{y}_1$$

$$\hat{y_2} - y = 0.952 - 0 = 0.952$$
$$\hat{y_2}(1 - \hat{y_2}) = 0.952(1 - 0.952) = 0.045$$
$$\nabla_\theta J^{(2)} = (0.952)(0.045) \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = 0.042 \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.0430 \\ 0.0860 \\ 0.0430 \end{bmatrix}$$

2.3. SGD update

We are using stochastic gradient descent (SGD) with a learning rate of 1.0. Recall that SGD updates the weights by using one datapoint at a time to compute the gradients. If we update the weights using the second example, what is the updated weight vector θ^1? Calculate $J(\theta^1)$

$$J(\theta^1) = \begin{bmatrix} -1.856 \\ 2.143 \\ 0.856 \end{bmatrix}$$

2.4. Inference

Given a new test example $x = [1, 3, 4]$, what output will the model produce?

$$\sigma[1, 3, 4] \begin{bmatrix} -1.856 \\ 2.143 \\ 0.856 \end{bmatrix} \approx 1$$

3 Programming Logistic Regression

Now write your own code for logistic regression. In the report, you need to include your code blocks for each task below and results and plots wherever appropriate. You can refer to the linear regression example notebook we used in class as a starting point (link is available in the slides). We will use the dataset from Scikit Learn `https://scikit-learn.org/stable/datasets/toy_dataset.html#breast-cancer-wisconsin-diagnostic-dataset`. For this programming assignment, our values will be the "mean radius" feature; output y will be 0 for malignant and 1 for benign. We will use the binary cross entropy loss function to train the model.

3.1. Divide the dataset

Write a python function to randomly split the dataset into 80% samples for training and 20% for testing. You will train the model on the training subset and once the model is trained, evaluate it on the testing subset.

```
def load_data() -> Tuple[NDArray, NDArray]:
    data: Any = load_breast_cancer()
    X: NDArray = data.data[:, 0].reshape(-1, 1)
    y: NDArray = data.target
    return X, y

def divide_dataset (data_x: NDArray,
```

```
                    data_y: NDArray,
                    train_percentage: float)
    n_entries: int = int(len(data_y)*train_percentage)
    train_data_x: NDArray = data_x[0:n_entries]
    train_data_y: NDArray = data_y[0:n_entries]
    test_data_x:  NDArray = data_x[n_entries:]
    test_data_y:  NDArray = data_y[n_entries:]
    return train_data_x , train_data_y , test_data_x , test_data_y
```

3.2. Sigmoid Function

Write a Python code for the sigmoid function. The input should be a Numpy array representing a batch of data. Each element in the array is an individual input to the sigmoid function. The output of sigmoid function should return a Numpy array of the same shape as the input.

```
def sigmoid_vec(X: np.ndarray) -> None:
    sigmoid_func = lambda x: 1 / (1 + math.exp(-x))
    for x_vec in X:
        for x_ind in range(len(x_vec)):
            X[x_ind] = sigmoid_func(X[x_ind])
```

3.3. Calculating Accuracy

For calculating accuracy, we will threshold the output, i.e., if $\hat{y} > 0.5$, classify as 1; otherwise 0. Write a function to calculate accuracy and print accuracy after every 1000th training iteration.

```
def calc_acc(y: np.ndarray, y_hat: np.ndarray) -> float:
    round_val = lambda val: 1 if val > 0.5 else 0
    num_entries = len(y); corr = 0
    for i in range(len(y)):
        y_hat_round = round_val(y_hat[i])
        if y[i] == y_hat_round: corr += 1
    return corr / num_entries
```

3.4. Weight Initialization

Using the inbuilt numpy.random, write a Python function to initialize the weights and biases.

```
def train(x: np.ndarray, y: np.ndarray, alpha: float, num_itters: int,
   log_mode: bool=False):
    # Initialize weights as a vector of shape (num_features,)
    num_features = x.shape[1]
    weight = np.random.uniform(0.0, 1.0, size=num_features)
    bias = 0.0
```

3.5. Gradient Descent

Given the specified loss function, write the equation for the gradient w.r.t. weight w and bias b. Then write a Python function to perform one step of gradient descent with learning rate a.

```
def gradient_descent(w: np.ndarray, b: float, alpha: float, X: np.ndarray,
    y: np.ndarray, y_hat: np.ndarray):
    num_entries = len(y)
```

```
    err = y_hat - y
    # Calculate average gradients
    grad_w = (1/num_entries) * np.dot(X.T, err)
    grad_b = (1/num_entries) * np.sum(err)
    # Update
    new_w = w - (alpha * grad_w)
    new_b = b - (alpha * grad_b)
    return new_w, new_b
```

3.6. Train the Model

Write a Python function for training the model using the template below:

```
def train(x: np.ndarray, y: np.ndarray, alpha: float, num_itters: int,
    log_mode: bool=False):
    # Initialize weights as a vector of shape (num_features,)
    num_features = x.shape[1]
    weight = np.random.uniform(0.0, 1.0, size=num_features)
    bias = 0.0
    # Training loop ------
    interval: int  = 1000
    accu_vals = []
    loss_vals = []
    epochs = 0
    for i in range(num_itters):
        y_hat = forward_pass(x, weight, bias)
        if (i%interval) == 0:
            # Record training accuracy after every 1000 iterations
            acc_val: float = calc_acc(y,y_hat)
            accu_vals.append(acc_val)
            # Record loss after every 1000 iterations
            loss_val = loss(y,y_hat)
            loss_vals.append(loss_val)
            print_evals(epochs, acc_val, loss_val)
            epochs += 1
        weight, bias = gradient_descent(weight, bias, alpha, x,y,y_hat)
    # Plot acc vs iterations and loss vs iterations
    plot_val(loss_vals, interval, "loss.png", "Loss v. Itter", "Itters", "
    Loss")
    plot_val(accu_vals, interval, "acc.png", "Accuracy v. Itters", "Itter"
    , "Accuracy")
    write_logs(accu_vals, interval, alpha)
    write_pretty_table("table.md")
    return weight, bias
```

Train the model with $\alpha = 0.1$ and 10000 iterations. The function should print training accuracy after every 1000 iterations. The function should return a plot of loss vs iterations and accuracy vs iterations. Show this plot in your report.

3.7. Write a test function to evaluate trained model

After training is done, you need to evaluate your model. Use the trained model to predict labels for the test data and calculate accuracy using the accuracy function. Print this accuracy.

```python
def test(test_x: np.ndarray, test_y: np.ndarray, weight: NDArray, bias:
    float):
    y_hat = forward_pass(test_x, weight, bias)
    acc_val: float = calc_acc(test_y, y_hat)
    y_hat = forward_pass(test_x, weight, bias)
    final_acc = calc_acc(test_y, y_hat)
    print(f"Training Complete. Fin Accuracy: {final_acc:.2%}")
```

3.8. Hyperparameter Analysis

Try 4 different learning rates a and 4 different number of iterations N. Report the test accuracy of the model for all combinations in a table like the one below:

α	**N0(0)**	**N1(1000)**	**N2(2000)**	**N3(3000)**
0.1	0.59121	0.77582	0.86813	0.87253
0.01	0.86154	0.87033	0.86813	0.87253
0.001	0.72527	0.81319	0.82857	0.83736
0.0001	0.59121	0.59121	0.59121	0.59121

With the best learning rate that you tried, does increasing the number of iterations from 10000 to 20000 help? Explain how you chose the values for learning rate and number of iterations.

3.9. (Extra Credit) Logistic Regression using Multiple Features

The Scikit-Learn dataset we used above has several features other than "mean radius". Train a model that uses more than one features in the input per example (mean radius and some other features). 139] To do so, you may have to rewrite the sigmoid function and gradient rules. Repeat the process and report your results. Did using more features help improve the accuracy on the test set? As there is more information, there is more concepts for the model to train on. However, this

α	**N0(0)**	**N1(1000)**	**N2(2000)**	**N3(3000)**
0.1	0.13407	0.98022	0.98242	0.98242
0.0001	0.14286	0.15824	0.17363	0.18681
0.001	0.12747	0.47473	0.92308	0.95385
0.001	0.13846	0.49231	0.87473	0.91648
0.01	0.09231	0.96044	0.97582	0.97802

Table 1: Your table caption here

also increases the overall complexity when it comes to resolving the underlying function during the training steps. As such there is a noteably sensitity to the Hyperparameters as opposed to the prior.

4 Image Convolution by Hand

Show the method and answer for convolving image $x(m,n)$ with filter $h(m,n)$. Assume zero-padding outside of x. The equation and visual procedure for 2D convolution is in the slides.

$$x(m,n) = X^{3\times 2} = \begin{bmatrix} 1 & 10 \\ 2 & 20 \\ 3 & 30 \end{bmatrix} \qquad h(m,n) = H^{2\times 3} = \begin{bmatrix} 1 & 0 & 1 \\ -1 & 0 & -1 \end{bmatrix}$$

$$X_{padded} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 10 & 0 & 0 \\ 0 & 0 & 2 & 20 & 0 & 0 \\ 0 & 0 & 3 & 30 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \qquad H_{flip} = \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix}$$

$$A = X * H := \begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 1 = a_{(0,0)}$$

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 10 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 10 = a_{(0,1)}$$

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 10 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 1 = a_{(0,2)}$$

$$\begin{bmatrix} 0 & 0 & 0 \\ 10 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 10 = a_{(0,3)}$$

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 1 = a_{(1,0)}$$

$$\begin{bmatrix} 0 & 1 & 10 \\ 0 & 2 & 20 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 10 = a_{(1,1)}$$

$$\begin{bmatrix} 1 & 10 & 0 \\ 2 & 20 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 1 = a_{(1,2)}$$

$$\begin{bmatrix} 10 & 0 & 0 \\ 20 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 10 = a_{(1,3)}$$

$$\begin{bmatrix} 0 & 0 & 2 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 1 = a_{(2,0)}$$

$$\begin{bmatrix} 0 & 2 & 20 \\ 0 & 3 & 30 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 10 = a_{(2,1)}$$

$$\begin{bmatrix} 2 & 20 & 0 \\ 3 & 30 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 1 = a_{(2,2)}$$

$$\begin{bmatrix} 20 & 0 & 0 \\ 30 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = 10 = a_{(2,3)}$$

$$\begin{bmatrix} 0 & 0 & 3 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = -3 = a_{(3,0)}$$

$$\begin{bmatrix} 0 & 3 & 30 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = -30 = a_{(3,1)}$$

$$\begin{bmatrix} 3 & 30 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = -3 = a_{(3,2)}$$

$$\begin{bmatrix} 30 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & -1 \\ 1 & 0 & 1 \end{bmatrix} = -30 = a_{(3,3)}$$

$$\begin{bmatrix} 1.0 & 10.0 & 1.0 & 10.0 \\ 1.0 & 10.0 & 1.0 & 10.0 \\ 1.0 & 10.0 & 1.0 & 10.0 \\ -3.0 & -30.0 & -3.0 & -30.0 \end{bmatrix}$$

5 Watch YouTube Videos as Homework :)

Watch the 2015 TED Talk delivered by Fei-Fei Li (Professor at Stanford and lead researcher behind the ImageNet dataset), titled "How we teach computers to understand pictures". Link: `https://youtu.be/40riCqvRoMs?feature=shared`.
Submit your responses to the following:

1. Summarize the video in a couple of paragraphs. You can write about the big-picture topic and specific projects (tasks, methods, results, etc.)
2. What was your favorite part of the talk / discussion?
3. Cite and summarize (in one paragraph) one of the speaker's published papers discussed in / relevant to the video.